UMRR11 T132 MSE V1.1.1 USER INTERFACE

DATE:
February 16, 2026

USER INTERFACE NAME:
UMRR11 T132 MSE

USER INTERFACE VERSION:
v1.1.1

s.m.s, smart microwave sensors GmbH
In den Waashainen 1
38108 Braunschweig
Germany

Phone: +49 531 39023-0
Fax: +49 531 39023-599
info@smartmicro.de
www.smartmicro.com

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

CONTENT

1 COMMUNICATION DATA STREAM SERVICE 2
1.1 COMTARGETLISTPORT PORT 2
1.2 COMOBJECTLISTPORT PORT 3
1.3 COMDYNAMICSPORT PORT 5
2 LEGAL DISCLAIMER NOTICE 7
APPENDIX 8
A COMMUNICATION DATA SERVICE API 8
B USER INTERFACE INSTRUCTIONS UMRR11 T132 MSE VERSION 1.1.1 17
B.1 Parameter Section auto_interface_0dim 17
B.2 Status Section auto_interface 23
B.3 Command Section auto_interface_command 25

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

1 COMMUNICATION DATA STREAM SERVICE

With the communication data stream service smartmicro ports can be received as C++ objects with simplified access functions, which are generated by the user interface. Smartmicro ports are data buffers which contains data recorded by the radar data: e.g objects, statistics, statuses of device etc. Each port contains a generic port header, with a port description : version, id, size etc. Sometime ports also contains dynamic list of objects. In order to receive a port, a callback needs to be registered with the service. The callback will be carried out periodically every sensor cycle time.

Please note:

- This callback will be called in the context of a receiver thread, so the data needs to be copied and the callback must be released. Otherwise, the reception will be blocked.
- It is possible to use one callback function for several clients with the same port and same user interface, but it is not allowed to use one callback function for different ports or different user interfaces.

For more details please see the examples below. The following ports are supported.

1.1 COMTARGETLISTPORT PORT

Description:
To receive the port called "ComTargetListPort" from a specific client, please use the following registration interface:

```
#include <umrr11_t132_mse_v1_1_1/DataStreamServiceIface.h>

void ReceiveComTargetListPortClbk(IN std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::
    comtargetlistport::ComTargetListPort> comTargetListPort, com::types::ClientId clientId)
{
        // Getting members of  ComTargetListPort
    std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::comtargetlistport::GenericPortHeader>
        genericPortHeader =
                        comTargetListPort->GetGenericPortHeader();
    std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::comtargetlistport::StaticPortHeader>
        staticPortHeader =
                        comTargetListPort->GetStaticPortHeader();
    auto targetList = comTargetListPort->GetTargetList();
    // Getting members of  GenericPortHeader
    std::cout << "Variable PortId:"
              << genericPortHeader->GetPortId()
              << std::endl;
    std::cout << "Variable PortVersionMajor:"
              << genericPortHeader->GetPortVersionMajor()
              << std::endl;
    std::cout << "Variable PortVersionMinor:"
              << genericPortHeader->GetPortVersionMinor()
              << std::endl;
    std::cout << "Variable Timestamp:"
              << genericPortHeader->GetTimestamp()
              << std::endl;
    std::cout << "Variable PortSize:"
              << genericPortHeader->GetPortSize()
              << std::endl;
    std::cout << "Variable BodyEndianness:"
              << genericPortHeader->GetBodyEndianness()
              << std::endl;
    std::cout << "Variable PortIndex:"
              << genericPortHeader->GetPortIndex()
              << std::endl;
    std::cout << "Variable HeaderVersionMajor:"
              << genericPortHeader->GetHeaderVersionMajor()
              << std::endl;
    std::cout << "Variable HeaderVersionMinor:"
              << genericPortHeader->GetHeaderVersionMinor()
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
            << std::endl;
        // Getting members of  StaticPortHeader
        std::cout << "Variable CycleTime:"
            << staticPortHeader->GetCycleTime()
            << std::endl;
        std::cout << "Variable NumberOfTargets:"
            << staticPortHeader->GetNumberOfTargets()
            << std::endl;
        // Getting members of  Target
        for(auto& target : targetList)
        {
            std::cout << "Variable Range:"
                << target->GetRange()
                << std::endl;
            std::cout << "Variable SpeedRadial:"
                << target->GetSpeedRadial()
                << std::endl;
            std::cout << "Variable AzimuthAngle:"
                << target->GetAzimuthAngle()
                << std::endl;
            std::cout << "Variable ElevationAngle:"
                << target->GetElevationAngle()
                << std::endl;
            std::cout << "Variable RCS:"
                << target->GetRCS()
                << std::endl;
            std::cout << "Variable Power:"
                << target->GetPower()
                << std::endl;
            std::cout << "Variable TgtNoise:"
                << target->GetTgtNoise()
                << std::endl;
        }

    }
        auto comDataStreamServ = com::master::umrr11_t132_mse_v1_1_1::DataStreamServiceIface::Get();
        ClientId clientIdA = 1024; // client id from sensor a
        ClientId clientIdB = 1025; // client id from sensor b
        ReceiveComTargetListPortCallback callback =
                            std::bind(&ReceiveComTargetListPortClbk,
                            std::placeholders::_1,
                            std::placeholders::_2);
        if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetListPortReceiveCallback(clientIdA,
            callback)
        {
            std::cout << "Failed to register ComTargetListPort port callback" << std::endl;
        }
        if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetListPortReceiveCallback(clientIdB,
            callback))
        {
            std::cout << "Failed to register ComTargetListPort port callback" << std::endl;
        }
```

1.2 COMOBJECTLISTPORT PORT

Description:
To receive the port called "ComObjectListPort" from a specific client, please use the following registration interface:

```
#include <umrr11_t132_mse_v1_1_1/DataStreamServiceIface.h>

void ReceiveComObjectListPortClbk(IN std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::
    comobjectlistport::ComObjectListPort> comObjectListPort, com::types::ClientId clientId)
{
        // Getting members of  ComObjectListPort
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::comobjectlistport::GenericPortHeader>
    genericPortHeader =
                    comObjectListPort->GetGenericPortHeader();
std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::comobjectlistport::StaticPortHeader>
    staticPortHeader =
                    comObjectListPort->GetStaticPortHeader();
auto objectList = comObjectListPort->GetObjectList();
// Getting members of  GenericPortHeader
std::cout << "Variable PortId:"
          << genericPortHeader->GetPortId()
          << std::endl;
std::cout << "Variable PortVersionMajor:"
          << genericPortHeader->GetPortVersionMajor()
          << std::endl;
std::cout << "Variable PortVersionMinor:"
          << genericPortHeader->GetPortVersionMinor()
          << std::endl;
std::cout << "Variable Timestamp:"
          << genericPortHeader->GetTimestamp()
          << std::endl;
std::cout << "Variable PortSize:"
          << genericPortHeader->GetPortSize()
          << std::endl;
std::cout << "Variable BodyEndianness:"
          << genericPortHeader->GetBodyEndianness()
          << std::endl;
std::cout << "Variable PortIndex:"
          << genericPortHeader->GetPortIndex()
          << std::endl;
std::cout << "Variable HeaderVersionMajor:"
          << genericPortHeader->GetHeaderVersionMajor()
          << std::endl;
std::cout << "Variable HeaderVersionMinor:"
          << genericPortHeader->GetHeaderVersionMinor()
          << std::endl;
// Getting members of  StaticPortHeader
std::cout << "Variable CycleTime:"
          << staticPortHeader->GetCycleTime()
          << std::endl;
std::cout << "Variable NumberOfObjects:"
          << staticPortHeader->GetNumberOfObjects()
          << std::endl;
std::cout << "Variable TimeStampOfMeasurement:"
          << staticPortHeader->GetTimeStampOfMeasurement()
          << std::endl;
// Getting members of  Object
for(auto& object : objectList)
{
    std::cout << "Variable xPos:"
              << object->GetxPos()
              << std::endl;
    std::cout << "Variable yPos:"
              << object->GetyPos()
              << std::endl;
    std::cout << "Variable zPos:"
              << object->GetzPos()
              << std::endl;
    std::cout << "Variable SpeedAbsolute:"
              << object->GetSpeedAbsolute()
              << std::endl;
    std::cout << "Variable Heading:"
              << object->GetHeading()
              << std::endl;
    std::cout << "Variable Length:"
              << object->GetLength()
              << std::endl;
    std::cout << "Variable Quality:"
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
                << object->GetQuality()
                << std::endl;
        std::cout << "Variable ObjectID:"
                << object->GetObjectID()
                << std::endl;
        std::cout << "Variable Status:"
                << object->GetStatus()
                << std::endl;
    }

}
    auto comDataStreamServ = com::master::umrr11_t132_mse_v1_1_1::DataStreamServiceIface::Get();
    ClientId clientIdA = 1024; // client id from sensor a
    ClientId clientIdB = 1025; // client id from sensor b
    ReceiveComObjectListPortCallback callback =
                        std::bind(&ReceiveComObjectListPortClbk,
                        std::placeholders::_1,
                        std::placeholders::_2);
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComObjectListPortReceiveCallback(clientIdA,
        callback)
    {
        std::cout << "Failed to register ComObjectListPort port callback" << std::endl;
    }
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComObjectListPortReceiveCallback(clientIdB,
        callback))
    {
        std::cout << "Failed to register ComObjectListPort port callback" << std::endl;
    }
```

1.3 COMDYNAMICSPORT PORT

Description:
To receive the port called "ComDynamicsPort" from a specific client, please use the following registration interface:

```
#include <umrr11_t132_mse_v1_1_1/DataStreamServiceIface.h>

void ReceiveComDynamicsPortClbk(IN std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::
    comdynamicsport::ComDynamicsPort> comDynamicsPort, com::types::ClientId clientId)
{
        // Getting members of  ComDynamicsPort
    std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::comdynamicsport::GenericPortHeader>
        genericPortHeader =
                        comDynamicsPort->GetGenericPortHeader();
    std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::comdynamicsport::StaticPortHeader>
        staticPortHeader =
                        comDynamicsPort->GetStaticPortHeader();
    // Getting members of  GenericPortHeader
    std::cout << "Variable PortId:"
              << genericPortHeader->GetPortId()
              << std::endl;
    std::cout << "Variable PortVersionMajor:"
              << genericPortHeader->GetPortVersionMajor()
              << std::endl;
    std::cout << "Variable PortVersionMinor:"
              << genericPortHeader->GetPortVersionMinor()
              << std::endl;
    std::cout << "Variable Timestamp:"
              << genericPortHeader->GetTimestamp()
              << std::endl;
    std::cout << "Variable PortSize:"
              << genericPortHeader->GetPortSize()
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
            << std::endl;
    std::cout << "Variable BodyEndianness:"
            << genericPortHeader->GetBodyEndianness()
            << std::endl;
    std::cout << "Variable PortIndex:"
            << genericPortHeader->GetPortIndex()
            << std::endl;
    std::cout << "Variable HeaderVersionMajor:"
            << genericPortHeader->GetHeaderVersionMajor()
            << std::endl;
    std::cout << "Variable HeaderVersionMinor:"
            << genericPortHeader->GetHeaderVersionMinor()
            << std::endl;
    // Getting members of  StaticPortHeader
    std::cout << "Variable DynamicsSource:"
            << staticPortHeader->GetDynamicsSource()
            << std::endl;
    std::cout << "Variable Speed:"
            << staticPortHeader->GetSpeed()
            << std::endl;
    std::cout << "Variable SpeedQuality:"
            << staticPortHeader->GetSpeedQuality()
            << std::endl;
    std::cout << "Variable YawRate:"
            << staticPortHeader->GetYawRate()
            << std::endl;
    std::cout << "Variable YawRateQuality:"
            << staticPortHeader->GetYawRateQuality()
            << std::endl;

}
    auto comDataStreamServ = com::master::umrr11_t132_mse_v1_1_1::DataStreamServiceIface::Get();
    ClientId clientIdA = 1024; // client id from sensor a
    ClientId clientIdB = 1025; // client id from sensor b
    ReceiveComDynamicsPortCallback callback =
                        std::bind(&ReceiveComDynamicsPortClbk,
                        std::placeholders::_1,
                        std::placeholders::_2);
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComDynamicsPortReceiveCallback(clientIdA,
        callback)
    {
        std::cout << "Failed to register ComDynamicsPort port callback" << std::endl;
    }
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComDynamicsPortReceiveCallback(clientIdB,
        callback))
    {
        std::cout << "Failed to register ComDynamicsPort port callback" << std::endl;
    }
```

For a more detailed API description, please see Appendix A.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

2 LEGAL DISCLAIMER NOTICE

All products, product specifications and data in this document may be subject to change without notice to improve reliability, function or otherwise. Not all products and/or product features may be available in all countries and regions. For legal reasons features may be deleted from products or smartmicro may refuse to offer products. Statements, technical information and recommendations contained herein are believed to be accurate as of the stated date. smartmicro disclaims any and all liability for any errors, inaccuracies or incompleteness contained in this document or in any other disclosure relating to the product.

To the extent permitted by applicable law, smartmicro disclaims (i) any and all liability arising out of the application or use of the product or the data contained herein, (ii) any and all liability of damages exceeding direct damages, including - without limitation - indirect, consequential or incidental damages, and (iii) any and all implied warranties, including warranties of the suitability of the product for particular purposes.

Statements regarding the suitability of products for certain types of applications are based on smartmicro's knowledge of typical requirements that are often placed on smartmicro products in generic/general applications. Statements about the suitability of products for a particular/specific application, however, are not binding. It is the customer's/user's responsibility to validate that the product with the specifications described is suitable for use in the particular/specific application. Parameters and the performance of products may deviate from statements made herein due to particular/specific applications and/or surroundings. Therefore, it is important that the customer/user has thoroughly tested the products and has understood the performance and limitations of the products before installing them for final applications or before their commercialization. Although products are well optimized to be used for the intended applications stated, it must also be understood by the customer/user that the detection probability may not be 100% and that the false alarm rate may not be zero.

The information provided, relates only to the specifically designated product and may not be applicable when the product is used in combination with other materials or in any process not defined herein. All operating parameters, including typical parameters, must be validated for each application by the customer's/user's technical experts. Customers using or selling smartmicro products for use in an application which is not expressly indicated do so at their own risk. This document does not expand or otherwise modify smartmicro's terms and conditions of purchase, including but not being limited to the warranty. Except as expressly indicated in writing by smartmicro, the products are not designed for use in medical, lifesaving or life-sustaining applications or for any other application in which the failure of the product could result in personal injury or death.

No license expressed or implied, by estoppel or otherwise, to any intellectual property rights is granted by this document or by any conduct of smartmicro. Product names and markings noted herein may be trademarks of their respective owners.

Please note that the application of the product may be subject to standards or other regulations that may vary from country to country. smartmicro does not guarantee that the use of products in the applications described herein will comply with such regulations in any country. It is the customer's/user's responsibility to ensure that the use and incorporation of products comply with regulatory requirements of their markets.

If any provision of this disclaimer is, or is found to be, void or unenforceable under applicable law, it will not affect the validity or enforceability of the other provisions of this disclaimer.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A COMMUNICATION DATA SERVICE API

A **ComTargetListPort**
Description: Provides a list of radar targets.
The object ComTargetListPort provides the following APIs:

```
std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::GenericPortHeader> GetGenericPortHeader
    () const;
```

Returns pointer to GenericPortHeader object, whose access functions are described below:
A **GenericPortHeader**
Description: No description available
The object GenericPortHeader provides the following APIs:

```
uint32_t GetPortId() const;
```

Returns value of PortId of uint32_t data type.
PortId - Port identification number.

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major version of the port API.

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor version of the port API.

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - Time of creation of the port.

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Size of the port including this header.

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Endianness of the sensor system.Please notice that the data will be presented in the host endianness, and should not be reversed.

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - The port index is used for multiple occurance of ports with the same port identifier.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Major version of the generic port API.

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Minor version of the generic port API.

```
std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::StaticPortHeader> GetStaticPortHeader()
    const;
```

Returns pointer to StaticPortHeader object, whose access functions are described below:
A **StaticPortHeader**
Description: Static header of the dynamics data port
The object StaticPortHeader provides the following APIs:

```
uint8_t GetDynamicsSource() const;
```

Returns value of DynamicsSource of uint8_t data type.
DynamicsSource - Dynamics Source

```
float32_t GetSpeed() const;
```

Returns value of Speed of float32_t data type.
Speed - Ego Speed

```
float32_t GetSpeedQuality() const;
```

Returns value of SpeedQuality of float32_t data type.
SpeedQuality - Ego Speed Quality

```
float32_t GetYawRate() const;
```

Returns value of YawRate of float32_t data type.
YawRate - Ego Yaw Rate

```
float32_t GetYawRateQuality() const;
```

Returns value of YawRateQuality of float32_t data type.
YawRateQuality - Ego Yaw Rate Quality

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
const std::vector<std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::Target>>&
    GetTargetList() const;
```

Returns pointer to array of Target objects, whose access functions are described below:
A **Target**
Description: Represents a single target
The object Target provides the following APIs:

```
float32_t GetRange() const;
```

Returns value of Range of float32_t data type.
Range - Target range

```
float32_t GetSpeedRadial() const;
```

Returns value of SpeedRadial of float32_t data type.
SpeedRadial - Target speed

```
float32_t GetAzimuthAngle() const;
```

Returns value of AzimuthAngle of float32_t data type.
AzimuthAngle - Azimuth angle of the target

```
float32_t GetElevationAngle() const;
```

Returns value of ElevationAngle of float32_t data type.
ElevationAngle - Elevation angle of the target

```
float32_t GetRCS() const;
```

Returns value of RCS of float32_t data type.
RCS - The Radar Cross Section of the target

```
float32_t GetPower() const;
```

Returns value of Power of float32_t data type.
Power - Amplitude of the target

```
float32_t GetTgtNoise() const;
```

Returns value of TgtNoise of float32_t data type.
TgtNoise - Noise of the target

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A **ComObjectListPort**
Description: Provides a list of tracked objects.
The object ComObjectListPort provides the following APIs:

```
std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::GenericPortHeader> GetGenericPortHeader
    () const;
```

Returns pointer to GenericPortHeader object, whose access functions are described below:
A **GenericPortHeader**
Description: No description available
The object GenericPortHeader provides the following APIs:

```
uint32_t GetPortId() const;
```

Returns value of PortId of uint32_t data type.
PortId - Port identification number.

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major version of the port API.

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor version of the port API.

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - Time of creation of the port.

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Size of the port including this header.

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Endianness of the sensor system.Please notice that the data will be presented in the host endianness, and should not be reversed.

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - The port index is used for multiple occurance of ports with the same port identifier.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Major version of the generic port API.

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Minor version of the generic port API.

```
std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::StaticPortHeader> GetStaticPortHeader()
    const;
```

Returns pointer to StaticPortHeader object, whose access functions are described below:
A **StaticPortHeader**
Description: Static header of the dynamics data port
The object StaticPortHeader provides the following APIs:

```
uint8_t GetDynamicsSource() const;
```

Returns value of DynamicsSource of uint8_t data type.
DynamicsSource - Dynamics Source

```
float32_t GetSpeed() const;
```

Returns value of Speed of float32_t data type.
Speed - Ego Speed

```
float32_t GetSpeedQuality() const;
```

Returns value of SpeedQuality of float32_t data type.
SpeedQuality - Ego Speed Quality

```
float32_t GetYawRate() const;
```

Returns value of YawRate of float32_t data type.
YawRate - Ego Yaw Rate

```
float32_t GetYawRateQuality() const;
```

Returns value of YawRateQuality of float32_t data type.
YawRateQuality - Ego Yaw Rate Quality

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
const std::vector<std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::Object>>&
    GetObjectList() const;
```

Returns pointer to array of Object objects, whose access functions are described below:
A **Object**
Description: Represents a single object
The object Object provides the following APIs:

```
float32_t GetxPos() const;
```

Returns value of xPos of float32_t data type.
xPos - X component of the position

```
float32_t GetyPos() const;
```

Returns value of yPos of float32_t data type.
yPos - Y component of the position

```
float32_t GetzPos() const;
```

Returns value of zPos of float32_t data type.
zPos - Z component of the position

```
float32_t GetSpeedAbsolute() const;
```

Returns value of SpeedAbsolute of float32_t data type.
SpeedAbsolute - Absolute object speed

```
float32_t GetHeading() const;
```

Returns value of Heading of float32_t data type.
Heading - Heading of an object

```
float32_t GetLength() const;
```

Returns value of Length of float32_t data type.
Length - Length of an object

```
float32_t GetQuality() const;
```

Returns value of Quality of float32_t data type.
Quality - Quality of an object

```
float32_t GetAcceleration() const;
```

Returns value of Acceleration of float32_t data type.
Acceleration - Acceleration of an object

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint16_t GetObjectID() const;
```

Returns value of ObjectID of uint16_t data type.
ObjectID - unique tag number for identification of the object

```
uint8_t GetStatus() const;
```

Returns value of Status of uint8_t data type.
Status - 0 = Object, 2 = Guardrail

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A **ComDynamicsPort**
Description: Provides the dynamics data used by the sensor.
The object ComDynamicsPort provides the following APIs:

```
std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::GenericPortHeader> GetGenericPortHeader
    () const;
```

Returns pointer to GenericPortHeader object, whose access functions are described below:
A **GenericPortHeader**
Description: No description available
The object GenericPortHeader provides the following APIs:

```
uint32_t GetPortId() const;
```

Returns value of PortId of uint32_t data type.
PortId - Port identification number.

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major version of the port API.

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor version of the port API.

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - Time of creation of the port.

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Size of the port including this header.

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Endianness of the sensor system.Please notice that the data will be presented in the host endianness, and should not be reversed.

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - The port index is used for multiple occurance of ports with the same port identifier.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Major version of the generic port API.

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Minor version of the generic port API.

```
std::shared_ptr<com::master::umrr11_t132_mse_v1_1_1::StaticPortHeader> GetStaticPortHeader()
    const;
```

Returns pointer to StaticPortHeader object, whose access functions are described below:
A **StaticPortHeader**
Description: Static header of the dynamics data port
The object StaticPortHeader provides the following APIs:

```
uint8_t GetDynamicsSource() const;
```

Returns value of DynamicsSource of uint8_t data type.
DynamicsSource - Dynamics Source

```
float32_t GetSpeed() const;
```

Returns value of Speed of float32_t data type.
Speed - Ego Speed

```
float32_t GetSpeedQuality() const;
```

Returns value of SpeedQuality of float32_t data type.
SpeedQuality - Ego Speed Quality

```
float32_t GetYawRate() const;
```

Returns value of YawRate of float32_t data type.
YawRate - Ego Yaw Rate

```
float32_t GetYawRateQuality() const;
```

Returns value of YawRateQuality of float32_t data type.
YawRateQuality - Ego Yaw Rate Quality

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

B USER INTERFACE INSTRUCTIONS UMRR11 T132 MSE VERSION 1.1.1

B.1 Parameter Section auto_interface_0dim

Automotive user interface 0dimensional parameters

Parameter Name	tx_antenna_idx
Description	Index of Transmit Antenna (0 and 1 are ACC, 2 is AEB)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	center_frequency_idx
Description	Index of center frequency (0..3 in ACC selectable. In AEB mode values 2 and 3 are similar to 1.)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	3

Parameter Name	frequency_sweep_idx
Description	Index of sweep (currently always 0 because only one sweep for every TX-antenna)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	0

Parameter Name	enable_tx_ant_toggle
Description	Automatic toggle of TX-Antenna 0->2->0 every radar cycle
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	1

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	angular_separation
Description	0 = Disable Angular Separation, 1 = Enable Angular Separation
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	1

Parameter Name	prf_selector_manual
Description	0 = PRF switching active, 1 = PRF index given in prf_selector_index is used
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	1

Parameter Name	prf_set_selector
Description	Select PRF set index (in manual or automatic PRF mode)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	1

Parameter Name	prf_manual_value_idx
Description	In manual PRF mode only: use nth element of selected set
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	azimuthal_angle_org
Description	[rad] Original Orientation of the sensor in Cartesian coordinate system in X-Y plane configured during setup, valid interval (-pi, pi)
Data Type	f32
Dimensions	None
Access	RW
Default	0
Min	-3.14159265
Max	3.14159265

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	elevation_angle_org
Description	[rad] Original Orientation of the sensor in Cartesian coordinate system in X-Z plane configured during setup, valid interval [-pi/2, pi/2]
Data Type	f32
Dimensions	None
Access	RW
Default	0
Min	-1.57079642
Max	1.57079642

Parameter Name	output_control_target_list_can
Description	send raw targets via CAN, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_object_list_can
Description	send objects via CAN, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_target_list_eth
Description	send raw targets via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_object_list_eth
Description	send objects via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	output_control_crash_barrier_can
Description	send crash barrier data via CAN, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_dynamics_eth
Description	send dynamics data via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	ip_source_address
Description	IP source address (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	3232238347

Parameter Name	subnet_mask
Description	Subnet mask (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	4294967040

Parameter Name	ip_dest_address
Description	IP destination address (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	3232238353

Parameter Name	ip_dest_port
Description	IP destination port
Data Type	u16
Dimensions	None
Access	RW
Default	55555

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	dynamics_source_switch
Description	0: Dynamics Data Source: dynSpdYawEst Port (ALGO), 1: Dynamics Data Source: pdrv Port (CAN), 2: Dynamics Data Source: pdrv Port (CAN) only speed and yawrate
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	sync_mode
Description	(Master+Slave config) 0=off, 1=master, 2=slave
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	sync_slave_identifier
Description	(Slave config) Unique Sync Slave Identifier, ignored on master (always 0)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	7

Parameter Name	sync_group_identifier
Description	(Slave config) Sync Group Identifier, ignored on master (always 0)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	1

Parameter Name	sync_nof_devices_1st_group
Description	(Master config) Number of synced devices (incl. master) in first group, ignored on slave
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	1
Max	8

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	sync_nof_devices_2nd_group
Description	(Master config) Number of synced devices in second group, ignored on slave
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	7

Parameter Name	time_sync_mode
Description	(Time Sync: Master+Slave config) 0=off, 1=master, 2=slave
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	time_sync_nof_devices
Description	(Time Sync: Master config) Number of time synced devices (incl. master), ignored on slave
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	1
Max	8

Parameter Name	track_min_vx_abs
Description	delete objects with an X-speed of -value<X-Speed<+value. If set to 0 nothing is deleted.
Data Type	f32
Dimensions	None
Access	RW
Default	2.77
Min	0
Max	14

Parameter Name	track_filter_behind_guardrail
Description	Filter leaving tracks reflections behind detected guardrail
Data Type	u32
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

B.2 Status Section auto_interface

customer status section

Status Name	auto_interface_version_major
Description	Automotive interface version major. Increased, if new version is not totally backward compatible.
Data Type	u32
Dimensions	None
Access	R

Status Name	auto_interface_version_minor
Description	Automotive interface version minor. Increased, if parameters or statuses are changed or added. The new version is still backward compatible.
Data Type	u32
Dimensions	None
Access	R

Status Name	sw_generation
Description	Software Version generation
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_major
Description	Software Version major
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_minor
Description	Software Version minor
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_patch
Description	Software Version patch
Data Type	u16
Dimensions	None
Access	R

Status Name	customer_id
Description	Customer Identifier
Data Type	u16
Dimensions	None
Access	R

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Status Name	product_serial
Description	32Bit product id serial
Data Type	u32
Dimensions	None
Access	R

Status Name	product_gen
Description	product generation
Data Type	u32
Dimensions	None
Access	R

Status Name	product_mod_high
Description	product modification high
Data Type	u32
Dimensions	None
Access	R

Status Name	product_mod_low
Description	product modification low
Data Type	u32
Dimensions	None
Access	R

Status Name	product_rev
Description	product revision
Data Type	u32
Dimensions	None
Access	R

Status Name	systime_low_dword
Description	time since the system timer was started in micro seconds (low dword)
Data Type	u32
Dimensions	None
Access	R

Status Name	systime_high_dword
Description	time since the system timer was started in micro seconds (high dword)
Data Type	u32
Dimensions	None
Access	R

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Status Name	acc_roll_angle_low_pass
Description	accelerometer angle of roll, 0 = accelerometer not calibrated
Data Type	f32
Dimensions	None
Access	R

Status Name	acc_pitch_angle_low_pass
Description	accelerometer angle of pitch, 0 = accelerometer not calibrated
Data Type	f32
Dimensions	None
Access	R

Status Name	acc_data_available
Description	accelerometer indication whether valid data is available
Data Type	u32
Dimensions	None
Access	R

B.3 Command Section auto_interface_command

Maintain compatible section 1000 commands

Command Name	comp_sensor_reset_delayed_app_start
Description	Perform a device reset and stay the given value [seconds] in the bootloader at next startup (3074.2)

Command Name	comp_fsm_core0_opmode
Description	Select top level FSM operation mode (3078.1)

Command Name	comp_eeprom_ctrl_factory_reset
Description	Performs factory reset (3075.4)

Command Name	comp_sensor_reset
Description	Reset command which starts from BIOS (if available) or bootloader (3074.1)

Command Name	comp_pdi_requestor_can
Description	Send PDI data to client (3076.1)

Command Name	comp_eeprom_ctrl_save_param_sec
Description	Save the parameter inside the EEPROM. (3075.3)

Command Name	comp_eeprom_ctrl_reset_param_sec
Description	Restore default values in RAM. EEPROM content is not changed. (3075.2)

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Command Name	comp_eeprom_ctrl_default_param_sec
Description	Restore default values in RAM and EEPROM. (3075.1)

Command Name	comp_timebase_set_seconds_val
Description	Set SECONDS value of NTP UTC timestamp

Command Name	comp_timebase_set_frac_seconds_val
Description	Set FRACTION_SECONDS value of NTP UTC timestamp

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.